# VitalFlow V1 — Billing & Revenue Cycle Management

Design + scaffold for the V1 billing domain. Covers the six core entities — **Charge**, **ChargeLine**, **Claim**, **Denial**, **Payment**, **PatientBalance** — plus the cross-cutting permissions, audit, and clearinghouse integration seam.

See also:

- docs/permissions-matrix.md — `billing:*` permissions
- docs/audit-logging.md — audit event taxonomy
- docs/clinical-domain.md — encounters, diagnoses (charge inputs)

---

## 1. Scope

End this step with these working end-to-end:

- charge capture from encounter outcomes
- claim record created from posted charges
- claim status tracking through adjudication
- denial queue (assignable work items)
- payment recording with patient/insurance source
- patient balance rollup + basic aging
- permissions + audit gating every write

---

## 2. Key design decisions

1. **Charge vs ChargeLine** — the existing `public.charges` table is already one CPT per row. V1 treats each row as a **ChargeLine**; **Charge (aggregate)** is the service-layer view of all charges for an encounter. No new table.

2. **Denial is its own table** — `claim_lines.denial_codes[]` captures the *fact* of a denial; queueable + assignable work items need a dedicated row with `assigned_to`, `status`, `priority`, `work_note`. One `denials` row per denied line (or per claim for claim-level denials).

3. **PatientBalance is cached** — dedicated `patient_balances` table (1 row per `(tenant_id, patient_id)` ), updated transactionally by `PaymentService.record` and `ChargeService.post`. Computing on-the-fly from invoices is correct but expensive on the billing dashboard.

4. **Clearinghouse is a seam, not a build** — V1 ships a `ClearinghouseSubmitter` interface. `claims.external_claim_id` + `claims.edi_envelope` persist after real submission. Default impl returns 501.

---

# 3. Entity-by-entity

## 3.1 Charge / ChargeLine

Uses existing `public.charges`.

**Lifecycle:** `draft → posted → billed → voided(terminal)`.

**Validations:**

- `cpt_code` OR `hcpcs_code` required
- `units > 0`, `unit_price_minor >= 0`
- `service_date <= today`
- `icd10_codes` non-empty at `posted` or later
- Cannot `posted → draft`
- Cannot `void` while on a submitted claim

**Service (`ChargeService`):** `listByEncounter`, `create`, `update`, `post`, `void`, `bulkCreateFromEncounter`.

**APIs:**

- `GET /api/v1/billing/encounters/:id/charges`
- `POST /api/v1/billing/charges`
- `PATCH /api/v1/billing/charges/:id`
- `POST /api/v1/billing/charges/:id/post`
- `POST /api/v1/billing/charges/:id/void`

**Edge cases:** duplicate CPT on same date is allowed (modifier-driven billing); voiding a charge tied to a paid claim line → 409; zero unit price posts with a warning.

**AC:** post without ICD-10 → 400; post emits `charge.created`; void of billed charge → 409.

## 3.2 Claim / ClaimLine

Uses existing `public.claims` + `public.claim_lines` + `public.claim_status_history`.

**Lifecycle:**

```
draft → ready → submitted → accepted → paid (terminal)
                                     ↘ partial → appealed → (resubmitted)
                                     ↘ rejected → (corrected → draft)
                                     ↘ denied → (appealed or written off)
any → closed (terminal)
```

**Validations:**

- Every `claim_line.charge_id` must reference a charge in `posted` status; submission flips those charges to `billed`
- `service_start_date <= service_end_date`
- `total_minor = SUM(claim_line.charge_minor)` (DB trigger)
- `paid_minor + patient_resp_minor <= total_minor`
- Unique charge across non-terminal claims (partial unique index)

**Service (`ClaimService`):** `list`, `getById`, `createFromCharges`, `markReady`, `submit`, `applyRemittance`, `appeal`, `close`. Every transition inserts a `claim_status_history` row.

**APIs:** eight endpoints under `/api/v1/billing/claims/*` — CRUD plus state transitions plus `/remittance`.

**Edge cases:** submit without clearinghouse → 501; partial payment with zero allowed → treat as line-level denial; double-billing a charge → 409.

**AC:** `createFromCharges` refuses non-posted charges; remittance with paid==total transitions to `paid` + emits `claim.status_changed`; history row per transition.

### 3.3 Denial (new)

**Schema:**

```
public.denials
├── id uuid PK
├── tenant_id uuid → tenants
├── claim_id uuid → claims
├── claim_line_id uuid → claim_lines  (nullable for claim-level denials)
├── denial_codes text[]
├── reason_text text
├── status text check in (open|working|appealed|resolved|written_off|uncollectable)
├── priority smallint default 3                 (1 = urgent)
├── assigned_to uuid → auth.users
├── assigned_at timestamptz
├── work_note text
├── resolution text
├── denied_amount_minor bigint
├── recovered_amount_minor bigint default 0
├── created_at, updated_at
```

**Lifecycle:** `open → working → appealed → resolved | written_off | uncollectable`.

**Validations:** `denied_amount_minor > 0`; `recovered_amount_minor <= denied_amount_minor`; `assigned_to` must have `billing:write`; terminal states cannot transition.

**Service (`DenialService`):** `list` (queue view — `open` + `working`, oldest-first), `createFromClaim`, `assign`, `recordWork`, `resolve`, `writeOff` (requires `billing:write_off`), `appeal`.

**APIs:** seven endpoints under `/api/v1/billing/denials/*`.

**Edge cases:** denial for a claim that later pays → auto-close with `resolution='paid_after_denial'`; assign to user without `billing:write` → 403.

**AC:** terminal state refuses transitions; queue stably ordered by priority asc, created_at asc; `applyRemittance` with zero allowed produces an `open` denial.

### 3.4 Payment

Uses existing `public.payments`.

**Lifecycle:** insert-only. Corrections = a second payment with negative `amount_minor` via `refund`.

**Validations:**

- Exactly one of `patient_id` or `payer_id` set (service-layer)
- `amount_minor != 0`
- `method` consistent with source (`insurance` needs `payer_id`; `cash/card/ach/check` need `patient_id`)
- `received_at <= now`

- Write recalculates `invoice.balance_minor` + `patient_balances` in the same transaction

**Service (`PaymentService`):** `record`, `list`, `refund`.

**APIs:** three endpoints under `/api/v1/billing/payments/*`.

**Edge cases:** overpayment → negative `invoice.balance_minor` → credit on `patient_balances`; ERA without invoice match → stored with `invoice_id=NULL` for manual posting; refund > original → 400.

**AC:** $X patient payment on $X invoice → `paid`; insurance payment emits `payment.recorded` with `payer_id`; refund creates a negative-amount row, does not delete the original.

### 3.5 PatientBalance (new)

**Schema:**

```
public.patient_balances
├── id uuid PK
├── tenant_id uuid → tenants
├── patient_id uuid → patients
│     unique (tenant_id, patient_id)
├── current_balance_minor bigint default 0       (can be negative = credit)
├── aging_0_30_minor bigint default 0
├── aging_31_60_minor bigint default 0
├── aging_61_90_minor bigint default 0
├── aging_over_90_minor bigint default 0
├── last_statement_at timestamptz
├── last_payment_at timestamptz
├── updated_at
```

**Lifecycle:** cached rollup, no state machine. Row is lazy-created on first charge/invoice/payment.

**Validations:** `aging_0_30 + aging_31_60 + aging_61_90 + aging_over_90 = current_balance_minor` (DB check); direct INSERT/UPDATE forbidden outside the service.

**Service (`PatientBalanceService`):** `get`, `list` (dashboard), `recalculate` (admin), `applyDelta` (called by Payment/Charge services).

**APIs:**

- `GET /api/v1/billing/patients/:id/balance`
- `POST /api/v1/billing/patients/:id/balance/recalculate`
- `GET /api/v1/billing/balances`

**Edge cases:** negative balance allowed, not auto-refunded; patient with no invoices returns zero row (don't insert until first event); aging buckets updated at statement time, interim only moves 0–30 bucket.

**AC:** $100 payment on $100 invoice → patient balance 0; `recalculate` matches `sum(invoice.balance_minor)` ; dashboard defaults to over-90 desc.

---

# 4. Cross-cutting

## 4.1 Permissions

| Capability | Permission |
|---|---|
| View everything | `billing:read` |
| Create/update charges, create/submit claims | `billing:write` |
| Record patient payments | `billing:collect` |
| Line adjustments, refunds | `billing:adjust` |
| Write-off (denial or balance) | `billing:write_off` |

Roles carrying these: `practice_owner` , `office_admin` , `biller` . Clinical roles ( `physician` , `nurse_ma` ) have none.

## 4.2 Audit events

Existing: `charge.created / updated / voided` , `invoice.issued / paid / voided` , `claim.submitted / status_changed / denied / appealed` , `payment.recorded / refunded` , `write_off.applied` .

Added in this slice:

- `denial.created`
- `denial.assigned`
- `denial.resolved`
- `patient_balance.recalculated`

## 4.3 Clearinghouse integration

`ClearinghouseSubmitter` interface (V1 = stub):

```
interface ClearinghouseSubmitter {
  submit837(claim: Claim, lines: ClaimLine[]): Promise<{ externalClaimId: string; ediEnvelop
  fetchStatus?(externalClaimId: string): Promise<{ status: ClaimStatus; raw: unknown }>;
  parse835?(ediPayload: string): Promise<Remittance>;
}
```

`ClaimService.submit()` accepts this dep. Default: `StubClearinghouseSubmitter` returns 501. Real impls (Availity / Change Healthcare / Claim.MD) land behind this interface — no downstream edits.

---

## 5. Acceptance criteria (slice-wide)

- [ ] `POST /billing/charges` creates a draft line; `.../post` transitions to posted and emits audit.
- [ ] `POST /billing/claims` with 3 posted charges creates a claim + 3 lines + initial history row.
- [ ] `POST /billing/claims/:id/submit` with stub clearinghouse returns 501; status unchanged.
- [ ] `POST /billing/claims/:id/remittance` with zero-allowed line creates an `open` denial.
- [ ] Denial queue endpoint returns `open` + `working`, paginated, priority-ordered.
- [ ] `POST /billing/payments` on a $100 invoice with $100 payment transitions invoice to `paid` and updates patient_balance to 0.
- [ ] `GET /billing/patients/:id/balance` returns zeros for a patient with no activity; non-zero after a posted charge.
- [ ] `GET /billing/balances` lists patients with outstanding balances, default sort over-90 desc.
- [ ] All writes emit the correct audit event; row-level triggers fire on denial / balance tables.
- [ ] RLS: no cross-tenant access on any billing table (verified by a test fixture).
- [ ] Typecheck + vitest pass monorepo-wide.

---

## 6. Not V1

- Real clearinghouse integration (Availity/CH/Claim.MD) — design seam exists, impl is next slice.
- Statement generation / printing / mailing.
- Payment plans + autopay.
- Collections workflow (dunning letters, bad-debt handoff).
- Contractual adjustments beyond line-level `adjustment_minor`.
- HCPCS Level II pricing tiers and fee schedules.
- Capitation / bundled-payment handling.
- Patient-portal bill pay (consumes these APIs later).